Programacion para Inteligencia Artificial

Clase completa: Scikit-learn, TensorFlow/Keras, PyTorch y aplicacion practica de Regresion Lineal

Alumno:	arg.gentile@gmail.com
Proyecto:	Programacion IA
Fecha:	Octubre 2026
Material base:	Modulo 4 - Bibliotecas de IA en Python

Indice de contenidos

BLOQUE 1 - Scikit-learn: Fundamentos del Machine Learning

1.1 Que es Scikit-learn y por que es el estandar

1.2 Conceptos previos: Que es Machine Learning

1.3 Los imports del arsenal

1.4 StandardScaler: el problema de las escalas

1.5 Variables categoricas: LabelEncoder y OneHotEncoder

1.6 Division train/test

BLOQUE 2 - Algoritmos de Clasificacion y Regresion

2.1 Regresion Logistica - Detector de Spam

2.2 Arboles de Decision - Prediccion de Abandono

2.3 Regresion Lineal - Prediccion de Engagement

2.4 Comparando algoritmos

BLOQUE 3 - TensorFlow y Keras: Redes Neuronales

3.1 El salto conceptual

3.2 Conceptos fundamentales

3.3 Primera red: Clasificador de resenias

3.4 Clasificacion multiclase y regresion

BLOQUE 4 - PyTorch: Control Flexible

BLOQUE 5 - APLICACION PRACTICA: Prediccion de Ventas

5.1 Diagnostico del problema

5.2 Que es la Regresion Lineal

5.3 Union de dos CSVs con pd.merge()

5.4 Ejemplo completo end-to-end

5.5 Interpretacion de resultados

5.6 Como adaptarlo a tus archivos reales

GLOSARIO DE CONCEPTOS CLAVE

BLOQUE 1 - Scikit-learn: Fundamentos del Machine Learning Tradicional

1.1 Que es Scikit-learn y por que es el estandar

Scikit-learn es **la biblioteca** por excelencia para machine learning tradicional en Python. Cuando decimos 'tradicional' nos referimos a algoritmos como regresion, arboles de decision, SVM, clustering - es decir, todo lo que NO es deep learning (eso lo veremos con TensorFlow y PyTorch).

Su gran virtud filosofica es la **interfaz uniforme**: no importa que modelo uses, todos hablan el mismo idioma:

- **.fit(X, y)** entrena el modelo
- **.predict(X)** genera predicciones
- **.score(X, y)** evalua el desempenio

Esto significa que si queres cambiar de regresion logistica a un random forest, cambias *una sola linea*. Esta portabilidad es un lujo cuando estas experimentando.

1.2 Conceptos previos: Que es Machine Learning

En programacion tradicional vos escribis las reglas: *'si el email contiene premio y viene de desconocido, es spam'*. En machine learning, le damos **ejemplos** al algoritmo (miles de emails ya etiquetados) y es el quien **descubre las reglas por su cuenta**.

Dos tipos de problemas fundamentales:

Tipo	Que predice	Ejemplo
Clasificacion	Una categoria discreta	Spam o no spam?
Regresion	Un valor numerico continuo	Cuanto costara esta casa?

1.3 Los imports del arsenal

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report
```

Analicemos por que cada uno:
- **numpy**: Scikit-learn trabaja internamente con arrays de NumPy
- **pandas**: para cargar y limpiar datos reales (CSV, Excel, etc.)
- **train_test_split**: parte los datos en entrenamiento y prueba
- **StandardScaler**: normaliza escalas de variables
- **LogisticRegression**: a pesar del nombre, es un algoritmo de **clasificacion**
- **DecisionTreeClassifier**: arboles de decision interpretables

- **accuracy_score y classification_report**: metricas de evaluacion

1.4 StandardScaler: el problema de las escalas

Supongamos que queremos predecir si un cliente comprara con dos variables: **edad** (18-70) e **ingreso** (50.000-500.000). Sin normalizar, el ingreso domina todos los calculos simplemente porque sus numeros son mas grandes. Es injusto.

La formula de estandarizacion:

```
z = (x - media) / desviacion_estandar
```

Esto deja a cada variable con **media 0 y desviacion estandar 1**.

```
from sklearn.preprocessing import StandardScaler
import numpy as np

datos = np.array([[25, 80000], [35, 120000], [45, 95000], [55, 150000]])
print("Datos originales:")
print(datos)
print(f"Media por columna: {datos.mean(axis=0)}")
print(f"Desv. estandar por columna: {datos.std(axis=0)}")

scaler = StandardScaler()
datos_normalizados = scaler.fit_transform(datos)
```

NOTA: Concepto crucial - fit vs transform: .fit_transform(X_train) calcula media y desviacion de los datos de entrenamiento Y transforma. .transform(X_test) aplica la MISMA transformacion aprendida sin recalcular. Por que? Porque los datos de prueba simulan datos del futuro: no podes recalcular media con datos que todavia no existen.

1.5 Variables categoricas: LabelEncoder y OneHotEncoder

a) LabelEncoder - para orden natural (bajo, medio, alto):

```
from sklearn.preprocessing import LabelEncoder

categorias = ['bajo', 'medio', 'alto', 'bajo', 'alto']
encoder = LabelEncoder()
categorias_codificadas = encoder.fit_transform(categorias)
# Resultado: [1, 2, 0, 1, 0] -&gt; alto=0, bajo=1, medio=2
```

Problema: LabelEncoder asigna numeros en orden alfabetico e introduce un orden numerico que puede confundir al modelo (es medio el doble de bajo?). Util solo cuando el orden tiene sentido.

b) OneHotEncoder - para categorias sin orden (colores, generos):

```
from sklearn.preprocessing import OneHotEncoder

colores = [['rojo'], ['azul'], ['verde'], ['rojo'], ['azul']]
encoder = OneHotEncoder(sparse_output=False)
colores_codificados = encoder.fit_transform(colores)
# rojo -&gt; [0, 1, 0]  (columnas: azul, rojo, verde)
```

Asi el modelo no asume que 'verde > rojo > azul'. Crea una columna binaria por cada categoria.

1.6 Division train/test - evitando el autoengano

Error clasico: entrenar y evaluar con los mismos datos. Es como estudiar con el examen enfrente: sacas 10 pero no aprendiste nada. El modelo **memoriza** en vez de aprender a generalizar.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)
```

Parametros clave:
- **test_size=0.25**: 25% a prueba, 75% a entrenamiento. Valores tipicos: 0.2 o 0.3
- **random_state=42**: fija la semilla aleatoria para que la division sea reproducible

BLOQUE 2 - Algoritmos de Clasificacion y Regresion

2.1 Regresion Logistica - Detector de Spam

A pesar del nombre, es un algoritmo de **clasificacion binaria**. Aprende a calcular la **probabilidad** de que algo pertenezca a una clase, y traza una frontera de decision.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report
import pandas as pd

datos_emails = {
    'cant_mayusculas': [45, 2, 38, 5, 3, 50, 8, 42, 6, 35],
    'cant_signos_exclamacion': [8, 0, 6, 1, 0, 10, 1, 7, 0, 5],
    'cant_urls': [5, 0, 4, 0, 1, 6, 0, 5, 0, 3],
    'longitud': [1500, 300, 1200, 450, 350, 1600, 400, 1300, 380, 1100],
    'es_spam': [1, 0, 1, 0, 0, 1, 0, 1, 0, 1]
}

df = pd.DataFrame(datos_emails)
X = df.drop('es_spam', axis=1)
y = df['es_spam']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

modelo = LogisticRegression()
modelo.fit(X_train, y_train)
predicciones = modelo.predict(X_test)

precision = accuracy_score(y_test, predicciones)
print(f"Precision: {precision:.2f}")

# Probamos con un email nuevo
nuevo_email = [[40, 7, 4, 1400]]
prediccion = modelo.predict(nuevo_email)
probabilidad = modelo.predict_proba(nuevo_email)
print(f"Confianza - Legitimo: {probabilidad[0][0]:.2%}, Spam: {probabilidad[0][1]:.2%}")
```

Puntos a destacar:
- **df.drop('es_spam', axis=1)**: axis=1 indica eliminar una **columna**
- **.predict()** devuelve la clase (0 o 1)
- **.predict_proba()** devuelve las probabilidades de cada clase

2.2 Arboles de Decision - Prediccion de Abandono de Clientes

Los arboles funcionan diferente: hacen **preguntas sucesivas de si/no** (como un diagrama de flujo). Su gran ventaja: **son interpretables**.

```
from sklearn.tree import DecisionTreeClassifier

# Arbol con profundidad maxima de 4 niveles (evita sobreajuste)
modelo = DecisionTreeClassifier(max_depth=4, random_state=42)
modelo.fit(X_train, y_train)
predicciones = modelo.predict(X_test)

# Importancia de cada caracteristica
importancias = pd.DataFrame({
```

```
    'caracteristica': X.columns,
    'importancia': modelo.feature_importances_
}).sort_values('importancia', ascending=False)
print(importancias)
```

> **NOTA: max_depth=4** limita la profundidad del arbol. Si no lo limitamos, el arbol crece hasta memorizar los datos -> sobreajuste (overfitting). Mas profundidad NO es mejor.

2.3 Regresion Lineal - Prediccion de Engagement en Redes

Cambiamos de problema: queremos predecir un **numero continuo** (likes). La regresion lineal busca la ecuacion: y = a1*x1 + a2*x2 + ... + c

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

modelo = LinearRegression()
modelo.fit(X_train, y_train)
predicciones = modelo.predict(X_test)

mse = mean_squared_error(y_test, predicciones)
r2 = r2_score(y_test, predicciones)
print(f"MSE: {mse:.2f}")
print(f"R2 score: {r2:.2f}")
```

Metricas de regresion:

- **MSE (Mean Squared Error)**: promedio de errores al cuadrado. Penaliza errores grandes
- **R2 score**: va de 0 a 1. 1.0 es perfecto, 0.0 es malo
- **.coef_**: los coeficientes aprendidos
- **.intercept_**: el valor base cuando todas las variables son cero

2.4 Comparando algoritmos - la ventaja de la interfaz uniforme

```
modelos = {
    'Regresion Logistica': LogisticRegression(random_state=42),
    'Arbol de Decision': DecisionTreeClassifier(random_state=42),
    'Random Forest': RandomForestClassifier(random_state=42)
}

for nombre, modelo in modelos.items():
    modelo.fit(X_train_scaled, y_train)
    y_pred = modelo.predict(X_test_scaled)
    precision = accuracy_score(y_test, y_pred)
    print(f"{nombre}: {precision:.4f}")
```

BLOQUE 3 - TensorFlow y Keras: Redes Neuronales

3.1 El salto conceptual

Las **redes neuronales** son modelos inspirados en el cerebro: capas de **neuronas artificiales** conectadas. Cada neurona recibe entradas, las procesa con una **funcion de activacion** y pasa el resultado a la siguiente capa.

3.2 Conceptos fundamentales

Funciones de activacion:
- **ReLU**: convierte negativos en 0, deja positivos iguales. La mas comun en capas ocultas
- **Sigmoid**: transforma cualquier valor en un numero entre 0 y 1. Para clasificacion binaria
- **Softmax**: para clasificacion multiclase. Convierte salidas en probabilidades que suman 1

Para entrenamiento:
- **Optimizador** (ej: adam): ajusta los pesos de la red
- **Funcion de perdida** (ej: binary_crossentropy): mide la distancia entre prediccion y realidad

3.3 Primera red: Clasificador de resenias

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

modelo = Sequential([
    Dense(64, activation='relu', input_shape=(4,)),
    Dense(32, activation='relu'),
    Dense(16, activation='relu'),
    Dense(1, activation='sigmoid')
])

modelo.compile(optimizer='adam',
               loss='binary_crossentropy',
               metrics=['accuracy'])

historia = modelo.fit(X_train, y_train,
                      epochs=50,
                      batch_size=32,
                      validation_split=0.2)

loss, accuracy = modelo.evaluate(X_test, y_test)
```

Conceptos del entrenamiento:
- **Epoch (epoca)**: una pasada completa por todos los datos. 50 epochs = 50 pasadas
- **batch_size=32**: procesa de a 32 ejemplos antes de actualizar pesos
- **validation_split=0.2**: reserva 20% para validacion durante entrenamiento

3.4 Clasificacion multiclase y regresion

Para **clasificacion multiclase**, cambiamos la salida y la perdida:

```
Dense(3, activation='softmax')   # 3 clases
modelo.compile(loss='categorical_crossentropy', ...)
```

Para **regresion** (predecir numeros continuos):

```
Dense(1)   # sin activacion, salida libre
modelo.compile(optimizer='adam', loss='mse', metrics=['mae'])
```

BLOQUE 4 - PyTorch: Control Flexible

PyTorch (desarrollada por Meta) ofrece **mas control y flexibilidad** que Keras. Es la favorita de los investigadores. Trabaja con **tensores**, similares a los arrays de NumPy pero optimizados para GPU.

```
import torch
import torch.nn as nn

class RedSimple(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super(RedSimple, self).__init__()
        self.capa1 = nn.Linear(input_size, hidden_size)
        self.relu = nn.ReLU()
        self.capa_salida = nn.Linear(hidden_size, output_size)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        x = self.capa1(x)
        x = self.relu(x)
        x = self.capa_salida(x)
        x = self.sigmoid(x)
        return x
```

Dos metodos clave:
- **__init__**: define las capas que tendra la red
- **forward**: define como fluyen los datos a traves de ellas

Aspecto	Keras	PyTorch
Facilidad	Muy alta	Media
Control	Medio	Muy alto
Ideal para	Prototipar rapido	Investigacion

BLOQUE 5 - APLICACION PRACTICA: Prediccion de Ventas

5.1 Diagnostico del problema

Tenemos dos CSVs: productos y ventas. Queremos predecir cantidad de stock futuro y probabilidad de vender un producto. Esto son **DOS problemas distintos**:

Lo que queres predecir	Tipo	Algoritmo
Cantidad de stock (numero: 50, 200 unidades)	Regresion	LinearRegression
Probabilidad de venta (si/no con %)	Clasificacion	LogisticRegression

NOTA: Error comun: querer resolver todo con regresion lineal. La regresion lineal da **numeros continuos**, no probabilidades entre 0 y 1. Para probabilidades necesitas regresion logistica (que a pesar del nombre, es clasificacion).

5.2 Que es la Regresion Lineal, intuitivamente

Imagina un grafico con precio en X y ventas en Y. Ves una nube de puntos. **La regresion lineal busca la mejor linea recta que pase por el medio de esa nube.**

Esa linea tiene una ecuacion:

```
ventas = a1 * precio + a2 * categoria + a3 * stock + ... + c
```

Los coeficientes ai y la constante c los **aprende el algoritmo** minimizando el error entre la linea y los puntos reales.

5.3 Union de dos CSVs con pd.merge()

Estructura tipica asumida:

```
productos.csv:
id_producto,nombre,categoria,precio,stock_actual
1,Remera Basica,Ropa,2500,120
2,Zapatillas Running,Calzado,15000,45

ventas.csv:
id_venta,id_producto,fecha,cantidad,descuento
101,1,2026-01-15,3,0
102,2,2026-01-15,1,10
```

Para unirlos:

```
datos = pd.merge(ventas, productos, on='id_producto', how='inner')
```

Parametros:
- **on='id_producto'**: la columna clave en comun
- **how='inner'**: solo registros que existan en AMBAS tablas
- **'left'** mantiene todos los de ventas, **'right'** todos los de productos, **'outer'** todos de ambos

5.4 Ejemplo completo end-to-end

Script completo con las dos predicciones:

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_squared_error, r2_score, accuracy_score

# 1) Cargar CSVs
productos = pd.read_csv('productos.csv')
ventas = pd.read_csv('ventas.csv', parse_dates=['fecha'])

# 2) Unir
datos = pd.merge(ventas, productos, on='id_producto', how='inner')

# 3) Feature engineering
datos['dia_semana'] = datos['fecha'].dt.dayofweek
datos['mes'] = datos['fecha'].dt.month
datos['es_finde'] = (datos['dia_semana'] &gt;= 5).astype(int)
datos['precio_final'] = datos['precio'] * (1 - datos['descuento']/100)

# 4) Agrupar ventas por producto y dia
ventas_diarias = datos.groupby(
    ['id_producto', datos['fecha'].dt.date, 'categoria',
     'precio', 'stock_actual']
).agg(unidades_vendidas=('cantidad', 'sum')).reset_index()

ventas_diarias['fecha'] = pd.to_datetime(ventas_diarias['fecha'])
ventas_diarias['dia_semana'] = ventas_diarias['fecha'].dt.dayofweek
ventas_diarias['mes'] = ventas_diarias['fecha'].dt.month
ventas_diarias['es_finde'] = (ventas_diarias['dia_semana'] &gt;= 5).astype(int)

# ---------- PROBLEMA 1: REGRESION ----------
features_num = ['precio', 'stock_actual', 'dia_semana', 'mes', 'es_finde']
features_cat = ['categoria']

X = ventas_diarias[features_num + features_cat]
y = ventas_diarias['unidades_vendidas']

preprocesador = ColumnTransformer([
    ('num', StandardScaler(), features_num),
    ('cat', OneHotEncoder(handle_unknown='ignore'), features_cat)
])

modelo_reg = Pipeline([
    ('preprocesador', preprocesador),
    ('regresor', LinearRegression())
])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

modelo_reg.fit(X_train, y_train)
predicciones = modelo_reg.predict(X_test)

print(f"MSE: {mean_squared_error(y_test, predicciones):.2f}")
print(f"R2:  {r2_score(y_test, predicciones):.3f}")

# Prediccion de stock futuro
nuevos = pd.DataFrame({
```

```
    'precio': [5000],
    'stock_actual': [100],
    'dia_semana': [5],  # sabado
    'mes': [3],
    'es_finde': [1],
    'categoria': ['Ropa']
})
stock_necesario = modelo_reg.predict(nuevos)
print(f"Stock estimado: {max(0, round(stock_necesario[0]))} unidades")

# ---------- PROBLEMA 2: CLASIFICACION ----------
umbral = ventas_diarias['unidades_vendidas'].median()
ventas_diarias['venta_alta'] = (
    ventas_diarias['unidades_vendidas'] &gt; umbral
).astype(int)

y_clf = ventas_diarias['venta_alta']

modelo_clf = Pipeline([
    ('preprocesador', preprocesador),
    ('clasificador', LogisticRegression(max_iter=1000))
])

X_train, X_test, y_train, y_test = train_test_split(
    X, y_clf, test_size=0.2, random_state=42)

modelo_clf.fit(X_train, y_train)

# Probabilidad de venta alta
probabilidades = modelo_clf.predict_proba(nuevos)
print(f"Prob. venta BAJA: {probabilidades[0][0]:.1%}")
print(f"Prob. venta ALTA: {probabilidades[0][1]:.1%}")
```

5.5 Interpretacion de resultados y Pipeline

El uso de **Pipeline** y **ColumnTransformer** es profesional: encadena preprocesamiento y modelo en un solo objeto, evitando bugs clasicos como olvidarse de aplicar el scaler a los datos nuevos.

Metricas interpretadas:

- **MAE = 3.10 unidades**: en promedio nos equivocamos en 3 unidades
- **R2 negativo**: si pasa, significa que el modelo es peor que predecir el promedio. Indica que los datos no tienen patron (o faltan variables predictivas)
- **Coeficientes**: muestran que variables aumentan o disminuyen la prediccion

5.6 Como adaptarlo a tus archivos reales

Reemplaza las primeras lineas por:

```
productos = pd.read_csv('/ruta/a/productos.csv')
ventas = pd.read_csv('/ruta/a/ventas.csv', parse_dates=['fecha'])
```

Si tus columnas se llaman diferente, ajusta: la clave de merge, los nombres en features_num y features_cat, y la variable objetivo.

Recomendaciones del profesor

Para predecir stock futuro (regresion):
1. Agrega variables temporales: estacionalidad, feriados, campanias
2. Agrega ventas historicas del mismo producto (variable muy predictiva)
3. Para series temporales largas, considera modelos como Prophet o ARIMA

Para probabilidad de venta (clasificacion):
1. La regresion logistica es un buen punto de partida
2. Para mejor precision prueba RandomForestClassifier o GradientBoostingClassifier - ganan la mayoria de competencias Kaggle para datos tabulares

GLOSARIO DE CONCEPTOS CLAVE

Array: Estructura de NumPy, elementos del mismo tipo, operaciones rapidas vectorizadas.

Batch: Grupo de ejemplos procesados juntos en una iteracion del entrenamiento.

Clasificacion: Tarea donde se predice una categoria. Binaria si hay solo dos clases.

ColumnTransformer: Herramienta de Scikit-learn para aplicar distintos preprocesamientos a distintas columnas.

Conjunto de prueba: Portion de datos reservada para evaluar el modelo final.

DataFrame: Estructura bidimensional de Pandas con filas y columnas etiquetadas.

Dropout: Tecnica anti-sobreajuste que apaga neuronas aleatoriamente durante entrenamiento.

Entrenamiento: Proceso donde el modelo aprende patrones ajustando sus parametros.

Epoca (epoch): Pasada completa del modelo por todos los datos de entrenamiento.

Estandarizacion: Transformar datos para tener media 0 y desviacion estandar 1.

Feature engineering: Crear nuevas caracteristicas derivadas a partir de las originales.

Funcion de perdida: Mide la distancia entre predicciones del modelo y valores reales.

LabelEncoder: Convierte etiquetas de texto a numeros enteros. Util para categorias con orden.

MAE (Mean Absolute Error): Promedio de las diferencias absolutas entre predichos y reales.

Matriz de confusion: Tabla de aciertos y errores por clase real vs clase predicha.

MSE (Mean Squared Error): Promedio de los cuadrados de las diferencias entre predichos y reales.

One-hot encoding: Convierte categorias en multiples columnas binarias, una por categoria.

Optimizador: Algoritmo que ajusta los pesos de la red para minimizar la perdida (ej: Adam).

Pipeline: Objeto de Scikit-learn que encadena preprocesamiento y modelo.

Precision (accuracy): Porcentaje de predicciones correctas sobre el total.

Regresion: Tarea donde se predice un valor numerico continuo.

ReLU: Funcion de activacion: cero para negativos, mismo valor para positivos.

R-cuadrado (R2): Metrica de regresion: proporcion de variabilidad explicada por el modelo.

Series: Estructura unidimensional de Pandas. Las columnas de un DataFrame son Series.

Sigmoid: Funcion de activacion que mapea cualquier valor a un rango entre 0 y 1.

Sobreajuste (overfitting): El modelo memoriza datos de entrenamiento pero falla con datos nuevos.

Softmax: Funcion de activacion para clasificacion multiclase. Salidas suman 1.

StandardScaler: Transformador de Scikit-learn que estandariza caracteristicas numericas.

Tensor: Estructura multidimensional similar a arrays de NumPy, base de deep learning.

Train-test split: Division de los datos en conjunto de entrenamiento y conjunto de prueba.

Cierre

El flujo profesional estandar en la industria combina estas herramientas:

```
Pandas (limpieza) -> NumPy (preprocesamiento) -> Scikit-learn o TensorFlow/PyTorch
(modelado)
```

Esta combinacion es la razon por la que Python se consolido como el lenguaje de la IA.

Material elaborado por: Prof. Claude - Programacion para Inteligencia Artificial